

Capstone workshop: map one of your real extract jobs

Day 2 (Tue Oct 6), 3:20–3:50 PM CT — in groups of 3–4, map one real DOM DoIT extract job onto the GCP target architecture. About 20 minutes to build, then each group walks the room through theirs.

Why you're doing this

This page is backlog draft zero. When your migration project kicks off, the map your group fills in below is where the backlog starts — so photograph it and email it to yourself before you leave. Everything else in this course was preparation; this is the artifact you take back to work.

How it works

1. Form a group of 3–4.
2. Pick **one** real extract job your team owns or operates. Not the biggest one, not the simplest one — the **most typical** one. A good heuristic: the one that pages someone when it fails.
3. Fill the template below, left to right. Paper, whiteboard, or a doc — the filled table is the deliverable.
4. Every box has an answer from the last two days. If your group gets stuck on a box, don't paper over it — flag it. A box you can't fill is the most valuable thing to raise in the share-out.
5. Keep it to one job. If you catch yourselves designing a platform, come back to the one job.

The mapping template (copy it and fill it in)

Row	Your answer
The job (name it)	<i>e.g. nightly extract of X to Y — one real job, with its real name</i>
Source system	<i>SQL Server table(s)? File drop? SaaS? Size? Change rate?</i>
Ingest tool	<i>Datastream (CDC) / Storage Transfer / BigQuery Data Transfer Service / bq load or LOAD DATA / gcloud — pick one, and say why</i>
Bronze (raw landing)	<i>GCS files (format? partition layout?) / external table / BigLake / Iceberg — what does raw look like for this job?</i>
Silver (cleaned/conformed)	<i>Dataform models: dedupe, types, conforming — what rules does this job's data need?</i>
Gold (mart)	<i>Which mart table(s)?</i>

Row	Your answer
	<i>Partitioned/clustered by what? Who queries it?</i>
Orchestration & CI/CD	<i>Dataform release config / Cloud Scheduler / Composer — and how does it deploy? (Your existing pipeline discipline → git → release configs)</i>
Serving	<i>Power BI — Import or DirectQuery for this mart? Refresh cadence tied to what event?</i>
First thing that breaks	<i>Honest answer: volume? schema drift? PII? the source system itself?</i>

Decision aids (when a box stalls you)

- **Is the source a database or files?** Database → **Datastream** (log-based CDC; SQL Server is a supported source) or a batch load, depending on freshness. Files → **Storage Transfer Service** to GCS, then **LOAD DATA** with a pinned schema — remember the autodetect N/A pitfall from Day 1: pin schemas, declare null markers, fail loudly instead of loading garbage. SaaS sources → **BigQuery Data Transfer Service**.
- **Nightly is fine, or always-fresh?** Nightly tolerance → batch load, it's simpler and loads are free. "Always current" → **Datastream CDC** — and the extract window disappears entirely.
- **Import or DirectQuery?** Default to **Import**, refreshed right after the pipeline event that rebuilds the mart ("after the Dataform release") — not a wall-clock guess. DirectQuery only with a stated intra-day freshness need. The full decision table is in the [Power BI connector guide](#).
- **Where do the transforms live?** Default: **Dataform**, in git, where your CI/CD discipline applies. Scheduled queries are fine for a single light derivation. Composer enters when there are many interdependent pipelines or non-BigQuery tasks — not on day one.

Examples to consult if you get stuck

Two filled-in maps. Peek at one if your group stalls — or finish yours first and compare afterward.

Example 1 — nightly case-records extract → Datastream CDC path

Row	Answer
The job	Nightly SQL Agent job extracts changed case records from the case-management SQL Server DB to a reporting copy; 2 AM window, occasionally collides with maintenance; downstream reports

Row	Answer
	stale after missed runs
Source system	SQL Server 2019, case tables, largest ~80M rows, steady daily churn (~1–2% of rows)
Ingest tool	Datastream — SQL Server is a supported source; log-based CDC, no triggers on production, no extract window. Backfill + continuous changes, landing in BigQuery
Bronze	Datastream-managed BigQuery tables (append-only change events / replicated tables) = raw-as-it-gets; optionally export snapshots to GCS Parquet for the file-lake copy
Silver	Dataform incremental models: latest-record-per-case (dedupe on the change stream), type cleanup, join to county/agency reference tables
Gold	mart_case_summary — partitioned by case date, clustered by county; feeds the existing case-load dashboards
Orchestration & CI/CD	Dataform release config on a schedule (Silver→Gold rebuild after Datastream lands); Datastream runs itself (M&O monitors, doesn't schedule); SQLX in git → existing PR/pipeline discipline
Serving	Power BI Import , refresh scheduled just after the Dataform release — matches the nightly-build reality
First thing that breaks	Schema drift on the source (a new column mid-release) — Datastream handles additive changes, but the Dataform models need an assertion to catch it

Notice: the 2 AM window disappears entirely, and M&O's role shifts from "restart the failed job" to "watch a stream's freshness metric."

Example 2 — county CSV file-drop → Storage Transfer + LOAD DATA path

Row	Answer
The job	Counties drop monthly CSV extracts (case/financial summaries) to an SFTP

Row	Answer
	share; an SSIS package sweeps the share, stages, cleans, loads
Source system	CSV files, ~40 counties, monthly, a few GB total; formats drift county to county; occasional late or re-sent files
Ingest tool	Files move to GCS (counties drop to a bucket, or Storage Transfer Service sweeps the existing share on a schedule); then LOAD DATA SQL with a pinned schema — never autodetect — into a Bronze table. A scheduled query with FORMAT_DATE-built URIs replaces the sweep
Bronze	GCS bucket, Hive layout county=X/period=YYYY-MM/ — original files kept immutable (replayable); BigLake table over the files for immediate query; loaded Bronze table for the conformed copy
Silver	Dataform: per-county staging models absorbing format drift (one model per variant), unioned and typed; assertions on row counts per county per period
Gold	mart_county_monthly — partitioned by period, clustered by county
Orchestration & CI/CD	Cloud Scheduler → scheduled query (load) → Dataform release (Silver→Gold); the whole flow is SQL + git — no SSIS package to version
Serving	Power BI Import , monthly refresh after the load window (DirectQuery is pointless — the data changes monthly)
First thing that breaks	A county changing its CSV layout without telling anyone — hence pinned schemas (the load fails loudly instead of loading garbage) and a Bronze assertion on unexpected NULL rates

Notice: this is the cheapest, fastest win in your estate — no CDC infrastructure, just a bucket + LOAD DATA + Dataform. And "late county re-sends file" stops being a rerun-the-package event: overwrite the file in the bucket, re-run one statement.

What good looks like — check your map before the share-out

- ☐ **One tool per box, not three.** A cell that says "Datastream or DTS or Dataflow" is a decision you deferred — acceptable for draft zero, but mark it as an open question so it survives.
- ☐ **Bronze stays raw and replayable.** No cleaning rules in Bronze. Bronze answers "what did the source say"; Silver answers "what does it mean." That separation is what makes reprocessing possible — the thing your current extract jobs can't do.
- ☐ **Dataform owns Silver→Gold, and it lives in git.** If transforms ended up somewhere else on your map (heavy scheduled queries, Power Query inside Power BI), you should be able to say why — legitimate answers exist, but the default belongs in Dataform where your CI/CD discipline applies.
- ☐ **Orchestration is boring.** Cloud Scheduler + Dataform release configs covers nearly everything you mapped today. Composer appears only if you can name the interdependency that needs it.
- ☐ **Power BI is Import-first.** DirectQuery appears only next to a stated freshness need, and the Serving row names the pipeline event the refresh follows.
- ☐ **The 'first thing that breaks' row is honest.** If it says "nothing breaks," you're not finished. Schema drift, PII, volume, a county that re-sends files — your real answers are the migration risk register's first entries.
- ☐ **It's specific.** Real table names, real counties, real row counts — that's what makes it backlog draft zero instead of a course exercise.

Before you leave

Photograph your map. Email it to yourself. When the migration project kicks off, this page is where the backlog starts.

Part of the Iowa DOM DoIT Data Engineering student pack — see ../00_schedule.md.